Name (Last, First): Answer Key

Section 02 Thursday, October 15, 2026, 2:40pm

This exam covers the exercises of Weeks 1 through 7, `00r_hello_rust` through `33k_paging`. It consists of 5 questions on 11 pages; be sure you have the entire exam before starting. Each question is worth 20 points, and each part's points are given at its start; the entire exam has 100 points.

This exam is closed book. The one reference you may use is the printed course Cheatsheet. No electronic devices of any kind. You may not share material with classmates during the exam. Each question prints the code it uses.

How to mark your answers. Most parts are answered in one of the three ways below. Each example is already answered, the way you should answer. Some parts of the exam also have one or two answers filled in, as examples.

1. **Circle one.** Circle the letter of your answer (or the word, where the options are words; "circle every" may take more than one).
   Which number is even? Circle one: **(A)** 3 **(B)** 4 **(C)** 7 **(D)** 9
2. **Fill in a blank.** Write your answer on the line. `0x10 + 0x10` = 0x20
3. **A numbered box in code.** Write the letter of the right option in the box, *and* circle that letter on the box's line of options.

```
fn double(x: u32) -> u32 {
    [1 B]
}
```

   Box 1: **(A)** `x + 2` **(B)** `x * 2` **(C)** `x / 2` **(D)** `x`

Changing an answer. Draw an X through the old circle, then circle your new answer:

Which number is even? Circle one: **(A)** 3 **(B)** 4 **(C)** 7 **(D)** 9

For a blank or a box, cross out what you wrote and write the new answer beside it; for a box, also change the circle on its line. If we cannot tell which answer is your final one, the part gets no credit.

Be concise and clearly indicate your answer. Answer each question in the space following the question. If you find it necessary to continue an answer elsewhere, indicate clearly the location of its continuation and label it with the question number and part. Where a part asks you to show your work, show it.

You should read through all the questions first, then pace yourself. If you need to ask a question during the exam, please raise your hand and I will come to your desk. Do not come to me.

The questions begin on the next page.

Problem	Possible	Score
1	20	
2	20	
3	20	
4	20	
5	20	
Total	100	

1. (_____/20 points)

Short Questions

(a) (5 points) Exercise 01r rounds addresses up to a page boundary:

```
pub const PAGE_SIZE: usize = 0x1000;
pub fn pgroundup(addr: usize) -> usize {
    (addr + PAGE_SIZE - 1) / PAGE_SIZE * PAGE_SIZE
}
pub fn checked_pgroundup(addr: usize) -> Option<usize> {
    match addr.checked_add(PAGE_SIZE - 1) {
        Some(bumped) => Some(bumped / PAGE_SIZE * PAGE_SIZE),
        None => None,
    }
}
```

(i) `pgroundup(0x8000_6FFF)` = 0x8000_7000 (ii) `pgroundup(0x8000_3000)` = 0x8000_3000

(iii) In a *debug* build, `pgroundup(usize::MAX - 2)` (circle one):
(A) panics **(B)** returns 0 **(C)** returns `None` **(D)** returns `usize::MAX`

(iv) In a *release* build (default settings), the same call (circle one):
(A) panics **(B)** returns 0 **(C)** returns `None` **(D)** returns `usize::MAX`

(v) `checked_pgroundup(usize::MAX - 2)` = None (checked_add: the add does not fit)

(b) (5 points) As in exercise 04r, `Pte` derives `Copy`. `Page` does not.

```
#[derive(Clone, Copy)]
pub struct Pte(pub u64);            // Copy: assigning a Pte copies it
pub struct Page { pub pa: usize }   // not Copy: assigning a Page moves it
pub fn release(page: Page) {}       // takes the Page by value

let p = Page { pa: 0x8000_3000 };   // 1
let a = Pte(0x2000_C001);           // 2
release(p);                         // 3
let b = a;                          // 4
let y = p.pa;                       // 5
let x = a.0;                        // 6
```

Exactly one of the numbered lines does not compile.

(i) Which line? 5 Why? Circle one:
(A) the variable is not declared `mut`
(B) `a` was moved by line 4
(C) its `Page` was moved out of that variable, so the variable can no longer be used
(D) `Page` is not `Copy`, so its fields can never be read

(ii) Line 6 reads `a` after line 4 assigned `a` to `b`. Why does it compile? Circle one:
(A) `Pte` is `Copy`: line 4 copied `a`
(B) `a` is never changed
(C) line 4 borrowed `a`
(D) `b` is never used

(iii) Suppose `Page` also had `#[derive(Clone, Copy)]`.
Which numbered lines would then not compile (or none)? none

(iv) With the line from (i) deleted, during which line is the `Page` dropped (the end of its life)? 3

(i) line 3 moved the Page from p into release (E0382). (iii) line 3 would copy p. (iv) release drops the Page when it returns.

(c) (5 points) Exercise 20a calls an assembly routine through this declaration. `extern "C"` passes the arguments in `a0`, `a1`, `a2`, in the order they are declared.

```
extern "C" { pub fn bytecopy(dst: *mut u8, src: *const u8, n: usize); }
bytecopy:                   # a0 = dst, a1 = src, a2 = n
    beqz a2, 2f             # if a2 == 0, go to the ret
1:  lb   t0, 0(a1)          # load one byte from the address in a1
    sb   t0, 0(a0)          # store it at the address in a0
    addi a0, a0, 1          # a0 = a0 + 1
    addi a1, a1, 1          # a1 = a1 + 1
    addi a2, a2, -1         # a2 = a2 - 1
    bnez a2, 1b             # if a2 != 0, go back to 1
2:  ret
```

A student changes the Rust declaration, but not the assembly, and makes a call that matches it:

```
pub fn bytecopy(dst: *mut u8, n: usize, src: *const u8);     unsafe { bytecopy(dst, 12, src) }
```

Here `src` is `0x8020_4000` and `dst` is `0x8020_6000`. There is no memory and no device at any address below `0x1000`.

(i) When `bytecopy`'s first instruction runs: `a0` = 0x8020_6000 `a1` = 12 (0xC) `a2` = 0x8020_4000

(ii) What happens? Circle one:

(A) the first `sb` stores to a small address and faults
(B) the 12 bytes at `src` are copied to `dst`, as intended
(C) the first `lb` loads from a small address and faults
(D) the 12 bytes at `dst` are copied to `src`

(iii) The kernel builds with no error and no warning. Why can the compiler not catch the mistake? Circle one:

(A) the linker puts the arguments back in order
(B) rustc checks each call against the declaration, but nothing checks the declaration against the assembly
(C) `unsafe` turns off type checking
(D) raw pointers carry no type

(d) (5 points) Exercise 06r keeps a process table as an array of `Option<u32>`: `None` is a free slot, and `Some(pid)` is a slot that holds process `pid`.

```
pub fn find_pid(table: &[Option<u32>], pid: u32) -> Option<usize> {
    for (index, slot) in table.iter().enumerate() {
        if *slot == Some(pid) {
            return Some(index);
        }
    }
    None
}

let table = [None, Some(4), Some(8), None, Some(2)];
```

(i) `find_pid(&table, 2)` = Some(4) (ii) `find_pid(&table, 3)` = None

(iii) `match find_pid(&table, 4) { Some(i) => i + 1, None => 0 }` = 2

(iv) `let i: usize = find_pid(&table, 2);` Circle one:

(A) does not compile: an `Option<usize>` is not a `usize`
(B) sets `i` to 4
(C) compiles, but panics when it runs
(D) sets `i` to 0

(v) `find_pid(&table, 3).unwrap()` (circle one):

(A) returns `None` **(B)** panics **(C)** does not compile **(D)** returns 0

(iii) Some(1) matches Some(i) with i = 1. (iv) E0308, mismatched types: take the value out with match, if let, or unwrap.

2. (_____/20 points)

Process States

Exercise 05r models a process's life as an enum. This is its solution, with a new event, `Kill`, for part (b). The numbered boxes are for part (b) and the blank line in `can_run` is for part (c).

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum ProcState {
    Unused,
    Runnable,
    Running,
    Sleeping { channel: u64 },
    Zombie { exit_status: i32 },
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Event {
    Alloc,                  // the kernel hands out this table slot
    Schedule,               // the scheduler picks this process
    Preempt,                // the timer takes the CPU back
    Sleep { channel: u64 }, // block until someone wakes this channel
    Wake { channel: u64 },  // a device or another process signals a channel
    Exit(i32),              // the process finishes, with a status
    Reap,                   // the parent collects the corpse, freeing the slot
    Kill,                   // part (b): someone kills the process
}

impl ProcState {
    pub fn can_run(&self) -> bool {
        match self {
            ProcState::Runnable | ProcState::Running => true,
            ProcState::Unused => false,
            ProcState::Sleeping { .. } => false,
            ProcState::Zombie { .. } => false,
            ProcState::Stopped => false,
        }
    }
}

pub fn next_state(state: ProcState, event: Event) -> Option<ProcState> {
    match (state, event) {
        (ProcState::Unused, Event::Alloc) => Some(ProcState::Runnable),
        (ProcState::Runnable, Event::Schedule) => Some(ProcState::Running),
        (ProcState::Running, Event::Preempt) => Some(ProcState::Runnable),
        (ProcState::Running, Event::Sleep { channel }) => Some(ProcState::Sleeping { channel }),
        (ProcState::Sleeping { channel: c }, Event::Wake { channel: w }) if c == w => {
            Some(ProcState::Runnable)
        }
        (ProcState::Running, Event::Exit(exit_status)) => Some(ProcState::Zombie { exit_status }),
        (ProcState::Zombie { .. }, Event::Reap) => Some(ProcState::Unused),
        (1 A                , Event::Kill) => Some(2 D                ),
        _ => None,
    }
}

pub fn apply(state: ProcState, event: Event) -> ProcState {
    match next_state(state, event) {
        Some(new_state) => new_state,
        None => state,
    }
}
```

(a) (8 points) Starting from `ProcState::Unused`, the kernel runs `state = apply(state, e)` for each event `e` below, in order. Give the state after each call, and write yes or no: did `apply` ignore the event because `next_state` returned `None`? Steps 1 and 4 are done for you.

Step	Event `e`	State after the call	Ignored?
1	`Alloc`	Runnable	no
2	`Preempt`	Runnable	yes, not Running
3	`Schedule`	Running	no
4	`Sleep { channel: 0x8 }`	Sleeping { channel: 0x8 }	no
5	`Wake { channel: 0x4 }`	Sleeping { channel: 0x8 }	yes, wrong channel
6	`Wake { channel: 0x8 }`	Runnable	no
7	`Schedule`	Running	no
8	`Exit(2)`	Zombie { exit_status: 2 }	no

(b) (6 points) Killing a process that is `Runnable`, `Running`, or `Sleeping` makes it `Zombie { exit_status: -1 }`. Killing an `Unused` slot or a `Zombie` is illegal.

(i) Write the letter of the right option in boxes 1 and 2 of the listing on the previous page.

Box 1: **(A)** `ProcState::Runnable | ProcState::Running | ProcState::Sleeping { .. }`
(B) `ProcState::Sleeping { channel }` **(C)** `_` **(D)** `ProcState::Running`

Box 2: **(A)** `ProcState::Zombie { exit_status }` **(B)** `ProcState::Runnable`
(C) `ProcState::Unused` **(D)** `ProcState::Zombie { exit_status: -1 }`

(ii) With your arm in place, what does `apply` return? The first is done for you.

`apply(ProcState::Runnable, Event::Kill)` = Zombie { exit_status: -1 }

`apply(ProcState::Unused, Event::Kill)` = Unused

(iii) Suppose the Kill arm were written *below* `_ => None` instead. Circle one:

(A) it works the same **(B)** it would not compile
(C) it compiles (with a warning), and every `Kill` is ignored **(D)** every event kills the process

(c) (6 points) Now add a state `ProcState::Stopped` (a process paused by the user, as with Ctrl-Z) and two events: `Stop` moves a `Running` process to `Stopped`, and `Continue` moves a `Stopped` process to `Runnable`.

(i) You add just the three new variants to the two enums and recompile. Which functions stop compiling? Circle one: **(A)** both **(B)** neither **(C)** `can_run` only **(D)** `next_state` only

(ii) Write the one line `can_run` needs, on the blank line in the listing.

(iii) Circle the *two* arms `next_state` needs for `Stop` and `Continue` (they go above `_ => None`):

(A) `(ProcState::Runnable, Event::Continue) => Some(ProcState::Stopped),`
(B) `(ProcState::Running, Event::Stop) => Some(ProcState::Stopped),`
(C) `(ProcState::Stopped, Event::Continue) => Some(ProcState::Runnable),`
(D) `(ProcState::Stopped, Event::Stop) => Some(ProcState::Running),`

(i) can_run has one arm per state: E0004, Stopped not covered; next_state's _ arm silently covers every new pair. (ii) _ => false also compiles, but hides the next new state.

3. (_____/20 points)

Scheduling Policies as Traits

Exercise 07r separates the scheduling *policy* from the loop that uses it:

```
#[derive(Clone, Copy, PartialEq, Eq, Debug)]
pub enum ProcState { Unused, Runnable, Sleeping }

pub trait Scheduler {
    fn pick_next(&mut self, states: &[ProcState]) -> Option<usize>;
    fn run_for(&mut self, states: &[ProcState], turns: usize) -> Vec<usize> {
        let mut picked = Vec::new();
        for _ in 0..turns {
            match self.pick_next(states) {
                Some(slot) => picked.push(slot),
                None => break,
            }
        }
        picked
    }
}

pub struct RoundRobin { pub next: usize }
impl RoundRobin {
    pub fn new() -> RoundRobin { RoundRobin { next: 0 } }
}
impl Scheduler for RoundRobin {
    fn pick_next(&mut self, states: &[ProcState]) -> Option<usize> {
        let n = states.len();
        for off in 0..n {
            let slot = (self.next + off) % n;
            if states[slot] == ProcState::Runnable {
                self.next = (slot + 1) % n;
                return Some(slot);
            }
        }
        None
    }
}
```

(a) (7 points) This code runs:

```
use ProcState::*;
let states = [Unused, Runnable, Sleeping, Runnable, Runnable];
let mut rr = RoundRobin::new();
let picked = rr.run_for(&states, 5);
```

Fill in the table for the five calls to `pick_next`. Calls 1 and 2 are done for you.

Call	`rr.next` before	Slots it looks at, in order	Returns	`rr.next` after
1	0	0, 1	Some(1)	2
2	2	2, 3	Some(3)	4
3	4	4	Some(4)	0
4	0	0, 1	Some(1)	2
5	2	2, 3	Some(3)	4

`picked` = [1, 3, 4, 1, 3]

(b) (5 points) The 07r file also has a second policy, `Priority`, and these two functions run any policy:

```
impl Scheduler for Priority { ... }   // runs the most urgent runnable slot
pub fn run_static<S: Scheduler>(sched: &mut S, states: &[ProcState]) -> Vec<usize> {
    sched.run_for(states, 3)
}
pub fn run_dyn(sched: &mut dyn Scheduler, states: &[ProcState]) -> Vec<usize> {
    sched.run_for(states, 3)
}
```

A program calls `run_static` four times: with a `Priority`, with a `RoundRobin`, with another `Priority`, and with another `RoundRobin`. It calls `run_dyn` twice: once with a `RoundRobin` and once with a `Priority`.

(i) Copies of `run_static` the compiler generates (circle one): **(A)** 1 **(B)** 2 **(C)** 3 **(D)** 4 one per type

(ii) Copies of `run_dyn` it generates (circle one): **(A)** 1 **(B)** 2 **(C)** 3 **(D)** 4

(iii) How does `run_dyn` call the right type's methods? Circle one:
(A) at run time: `&mut dyn Scheduler` carries a pointer to a table of methods, a vtable (dynamic dispatch)
(B) at compile time: the compiler makes a copy of `run_dyn` for each type (static dispatch)

(c) (8 points) `Backward` is a policy that rotates the other way. Each call looks at the slots going *down* from `self.next` (`next`, `next - 1`, …, `0`, then wrapping around to the last slot) and takes the first `Runnable` one. Each time it looks at a slot, it steps the cursor down one slot, wrapping from 0 to the last slot. `Backward::new()` starts with `next` = 0.

```
impl Scheduler for Backward {
    fn pick_next(&mut self, states: &[ProcState]) -> Option<usize> {
        let n = states.len();
        for _ in 0..n {
            let slot = self.next;
            if self.next == 0 {
                self.next = [1 A          ];
            } else {
                self.next = [2 B          ];
            }
            if states[slot] == ProcState::Runnable {
                return Some(slot);
            }
        }
        None
    }
}
```

(i) Write the letter of the right option in boxes 1 and 2, and circle it on that box's line.
Box 1: **(A)** `n - 1` **(B)** `self.next - 1` **(C)** `n` **(D)** `0`
Box 2: **(A)** `slot + 1` **(B)** `self.next - 1` **(C)** `self.next + 1` **(D)** `n - 1`

(ii) Why does the code test `self.next == 0` instead of always using `self.next - 1`? Circle one:
(A) slot 0 is never `Runnable`
(B) without it, the scan would go up instead of down
(C) when `self.next` is 0, `self.next - 1` underflows: a debug build panics
(D) `-` cannot be used on a `usize`

(iii) For part (a)'s `states`, `Backward::new().run_for(&states, 4)` returns [4, 3, 1, 4]
Show your work: for each call, the slots it looks at and the slot it takes.

call 1 (next 0): looks at 0, 4 → takes 4, next = 3. call 2 (next 3): 3 → takes 3, next = 2.
call 3 (next 2): 2, 1 → takes 1, next = 0. call 4 (next 0): 0, 4 → takes 4, next = 3.

4. (_____/20 points)

Streaming with O(1) State

This is `count_fd` from the 12c `wc` solution. `ulib::read(fd, &mut buf)` reads up to `buf.len()` bytes into `buf` and returns `Ok(n)`, the number it read, which may be fewer than asked for; `Ok(0)` means end of file.

```
#[derive(Default, Clone, Copy)]
struct Counts { lines: usize, words: usize, bytes: usize }

fn is_space(b: u8) -> bool {
    matches!(b, b' ' | b'\t' | b'\n' | b'\r' | 0x0b | 0x0c)
}

fn count_fd(fd: Fd) -> Result<Counts, ulib::Error> {
    let mut c = Counts::default();
    let mut in_word = false;
    let mut buf = [0u8; 512];
    loop {
        let n = ulib::read(fd, &mut buf)?;
        if n == 0 {
            return Ok(c);
        }
        for &b in &buf[..n] {
            c.bytes += 1;
            if b == b'\n' {
                c.lines += 1;
            }
            if is_space(b) {
                in_word = false;
            } else if !in_word {
                in_word = true;
                c.words += 1;
            }
        }
    }
}
```

(a) (6 points) `count_fd(STDIN)` runs, and its calls to `ulib::read` return the bytes below (␣ is a space, `\n` a newline). Fill in the table with the values after each call has been handled, then give what `count_fd` returns. Call 1 is done for you.

Call	Bytes read	`n`	`c.lines`	`c.words`	`c.bytes`	`in_word`
1	one␣t	5	0	2	5	true
2	wo\nx	4	1	3	9	true
3	y␣\n	3	2	3	12	false
4	end of file	0	2	3	12	false

`count_fd` returns: Ok(Counts { lines: 2, words: 3, bytes: 12 })

(b) (4 points) A student moves `let mut in_word = false;` from above the `loop` to the first line *inside* it.

(i) For the input in part (a), the student's `count_fd` returns lines 2 words 5 bytes 12

(ii) All five of wc's tests still pass. Their inputs are short, and the test harness answers each `read` with as many of the remaining bytes as fit in the buffer. Why do the tests not catch the bug? Circle one:

(A) the tests check only the byte count
(B) `in_word` is reset at every newline anyway
(C) each test input arrives in one `read`, so no word is split across two reads
(D) the tests run in a release build

(c) (4 points) This is `cat_fd` from the 11c `cat` solution. `ulib::write_all(fd, bytes)` writes every byte of `bytes`.

```
fn cat_fd(fd: Fd) -> Result<(), ulib::Error> {
    let mut buf = [0u8; 512];
    loop {
        let n = ulib::read(fd, &mut buf)?;
        if n == 0 { return Ok(()); }          // end of file
        ulib::write_all(STDOUT, &buf[..n])?;
    }
}
```

`cat_fd` copies a 1200-byte file. How many calls does it make to `read` and to `write_all`?

(i) If each `read` returns 512 bytes (or what remains): `read` 4 `write_all` 3 512, 512, 176, 0

(ii) If each `read` returns 200 bytes (or what remains): `read` 7 `write_all` 6 200 x 6, 0

(d) (6 points) Using `count_fd` as a model, `longest_word` returns the length in bytes of the longest word read from `fd`. As in `count_fd`, a word is a run of bytes that are not `is_space`. A last word with nothing after it still counts. For example, `ab␣cdef\ng\n` gives 4. Words may be split across reads. Box 1 is done for you. Write the letter of the right option in boxes 2 and 3, and circle it on that box's line.

```
fn longest_word(fd: Fd) -> Result<usize, ulib::Error> {
    let mut buf = [0u8; 512];
    let mut cur = 0;          // the length of the word we are in, so far
    let mut longest = 0;      // the longest seen
    loop {
        let n = ulib::read(fd, &mut buf)?;
        if n == 0 {                       // a last word with nothing after it still counts
            if cur > longest { longest = cur; }
            return Ok(longest);
        }
        for &b in &buf[..n] {
            if [1 B] {
                if cur > longest { longest = cur; }
                [2 C];
            } else {
                [3 B];
            }
        }
    }
}
```

Box 1: **(A)** `b == b'\n'` **(B)** `is_space(b)` **(C)** `b == b' '` **(D)** `b != b' '`

Box 2: **(A)** `longest += 1` **(B)** `cur += 1` **(C)** `cur = 0` **(D)** `longest = 0`

Box 3: **(A)** `longest += 1` **(B)** `cur += 1` **(C)** `cur = 0` **(D)** `longest = 0`

cur lives outside the loop, like in_word, so a word split across reads keeps counting.

5. (_____/20 points)

The Free List and a Page Table

Exercise 32k's page allocator, and the page-table entries of exercise 33k.

```
pub const PGSIZE: usize = 4096;    // 0x1000
pub const PTE_V: usize = 1; pub const PTE_R: usize = 2; pub const PTE_W: usize = 4; pub const PTE_X: usize = 8;

#[repr(C)]
struct Run { next: *mut Run }      // a free page's first 8 bytes
static mut FREELIST: *mut Run = ptr::null_mut();

pub unsafe fn kfree(pa: *mut u8) {
    let r = pa as *mut Run;
    (*r).next = FREELIST;
    FREELIST = r;
}
pub unsafe fn kalloc() -> *mut u8 {
    let r = FREELIST;
    if !r.is_null() { FREELIST = (*r).next; }
    r as *mut u8
}

#[derive(Clone, Copy)] pub struct Pte(pub usize);         // one page-table entry
impl Pte {
    pub const fn new(pa: usize, flags: usize) -> Pte { Pte(((pa >> 12) << 10) | flags) }
    pub const fn pa(self) -> usize                   { (self.0 >> 10) << 12 }
    pub const fn is_valid(self) -> bool              { self.0 & PTE_V != 0 }
}
```

An Sv39 virtual address is three 9-bit indices over a 12-bit offset, and a PTE packs a physical page number above ten bits of flags:

```
 38        30 29        21 20        12 11          0
+------------+------------+------------+-------------+
|   VPN[2]   |   VPN[1]   |   VPN[0]   |   offset    |      VPN[i] = (va >> (12 + 9 * i)) & 0x1FF
+------------+------------+------------+-------------+

 63    54 53             10 9 8 7 6 5 4 3 2 1 0
+--------+-----------------+---+-+-+-+-+-+-+-+-+          flag values:  V = 1    valid
|reserved|  PPN (44 bits)  |RSW|D|A|G|U|X|W|R|V|                        R = 2    read
+--------+-----------------+---+-+-+-+-+-+-+-+-+                        W = 4    write
                                                                         X = 8    execute
```

A page table is one 4096-byte page holding 512 PTEs of 8 bytes each. VPN[2] picks an entry of the root table, VPN[1] an entry of a level-1 table, and VPN[0] an entry of a level-0 table. A valid PTE with R, W and X all clear is a *branch*: its address is the next table. A valid PTE with any of R, W, X set is a *leaf*: its address is the mapped page. For a page-aligned `pa`, `((pa >> 12) << 10)` is just `pa / 4`: for example, `0x8000_4000 / 4` = `0x2000_1000`.

`map_page(root, va, pa, perm)` maps one virtual page, as 33k's `mappages` does. It starts at the root table. At level 2, and then at level 1, it looks at entry VPN[level] of the table. If that entry is not valid, it takes a page with `kalloc()`, fills it with zeros, and writes the branch PTE `Pte::new(page, PTE_V)` into the entry. Either way, it follows the entry down to the next table. At level 0 it writes the leaf PTE `Pte::new(pa, perm | PTE_V)` into entry VPN[0].

(a) (5 points) The free list starts as

```
FREELIST -> 0x8040_C000 -> 0x8040_8000 -> 0x8041_0000 -> 0x8040_4000 -> null
```

The kernel runs the four lines below, in order. Give what each returns, and the value of `FREELIST` (the first page on the list) after it. The first line is done for you.

Line	returns	`FREELIST` afterwards
`let a = kalloc();`	0x8040_C000	0x8040_8000
`let b = kalloc();`	0x8040_8000	0x8041_0000
`kfree(a);`	(nothing)	0x8040_C000
`let c = kalloc();`	0x8040_C000	0x8041_0000

Right after `kfree(a)`, the first 8 bytes of page `a` hold 0x8041_0000

(b) (8 points) Continuing from part (a), the kernel fills page `b` with zeros, makes it the root table, `root = b as *mut Pte`, and calls

```
map_page(root, 0x0080_5000, 0x8030_0000, PTE_R | PTE_X)
```

The virtual page `0x0080_5000` has VPN[2] = 0, VPN[1] = 4, and VPN[0] = 5. Fill in the three PTEs the call writes. The first row is done for you.

Level	Table address	Index	Value written	Branch or leaf	It points at
2 (root)	0x8040_8000	0	0x2010_4001	branch	0x8041_0000
1	0x8041_0000	4	0x2010_1001	branch	0x8040_4000
0	0x8040_4000	5	0x200C_000B	leaf	0x8030_0000

After the call, `FREELIST` = null

walk's two kallocs took the rest of the list: 1_0000, then 4000. Leaf: 0x8030_0000 / 4 = 0x200C_0000, OR (R + X + V = 0xB).

(c) (3 points) Use the tables of part (b).

(i) The virtual address `0x0080_5123` translates to 0x8030_0123 leaf page + offset 0x123

(ii) A program uses the virtual address `0x0080_6000`. Its VPN[2], VPN[1], VPN[0] are 0, 4, 6. What happens? Circle one:

(A) it translates to `0x8030_1000`, the next physical page
(B) the MMU takes a page with `kalloc()` and maps it
(C) a page fault: entry 6 of the level-0 table is not valid
(D) it translates to `0x8030_0000`, the same page

(d) (4 points) Work these by hand.

(i) A PTE holds `0x2001_0007`. Circle every flag that is set: (V) (R) (W) X 0x7 = 4 + 2 + 1

(ii) That PTE's address, `pa()`, is 0x8004_0000 (iii) It is a (circle one): branch (leaf)

(iv) A table is at `0x8040_6000`; each PTE is 8 bytes. Entry 5 is at 0x8040_6028 5 x 8 = 0x28